Motion Timeline Editor — Technical Documentation

Oct 23, 2026 · @Syed Jafri

Overview

Motion Timeline Editor is a browser-based keyframe editor for UI and vector motion: animate layer properties on a timeline, shape timing with editable cubic-bezier curves, review motion with onion-skinning, and export the result as Lottie JSON or plain CSS `@keyframes`.

Who it is for — product designers prototyping interface motion, and front-end engineers who need production-ready animation code without hand-tuning timing values.

Key capabilities

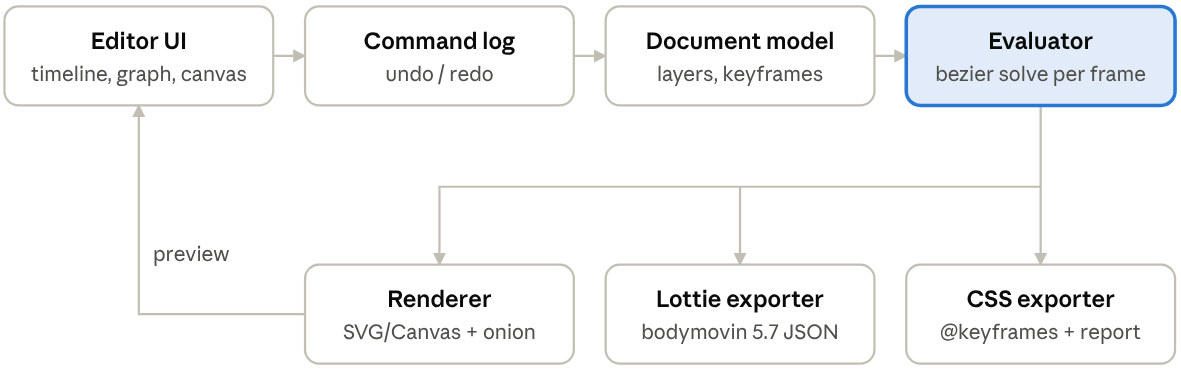
- Layered scene of shapes, groups, text and imported SVG, each with animatable transform, opacity, fill, stroke and path properties.
- Per-property keyframe tracks with hold, linear and bezier interpolation.
- Graph editor with draggable bezier handles, numeric entry, presets and copy/paste of `cubic-bezier()` values.
- Onion-skinning of past and future frames with configurable range, step and tint.
- Exporters: Lottie (bodymovin schema 5.x) and CSS keyframes (one `@keyframes` block per animated element), with a fidelity report for anything that cannot be represented.

Out of scope (v1) — raster/bitmap animation, audio sync, expressions, 3D layers, and Lottie import of arbitrary After Effects files (only files this tool exported round-trip).

Architecture

Edits flow one way into the document model; a single evaluator turns the model into scene state for the on-screen preview and for both exporters.

One evaluator drives preview and both exporters



system architecture · edit, evaluate, render, export

Stack: TypeScript throughout; `@motion-tl/core` (model, evaluator, exporters, no DOM dependency), a React editor app, SVG and Canvas 2D renderers, and lottie-web only in tests for parity checks. State lives in an immutable store; the evaluator memoizes per (track, frame).

Document model

A project is a tree of layers; every animatable property owns an ordered list of keyframes, and time is stored in integer frames at the composition's frame rate so edits never accumulate float drift.

Hierarchy — Composition → Layers (shape, group, text, svg, null) → Properties → Keyframes. Groups and nulls can parent other layers; child transforms compose with the parent's.

Animatable properties

Property	Type	Interpolated as
`position`	vec2 (px)	Per component, optional spatial bezier path
`anchor`	vec2 (px)	Per component
`scale`	vec2 (%)	Per component
`rotation`	number (deg)	Linear in degrees, unbounded (720° = two turns)
`opacity`	number (0–100)	Clamped after easing

Property	Type	Interpolated as
`fill`, `stroke`	color (RGBA)	In linear sRGB, then re-encoded
`strokeWidth`	number (px)	Scalar
`path`	bezier path	Vertex-wise; keyframes must share vertex count
`trimStart`, `trimEnd`	number (%)	Scalar

Keyframe

```
interface Keyframe<T> {
  id: string;
  frame: number;                // integer, composition frames
  value: T;
  interp: "hold" | "linear" | "bezier";
  easeOut: [number, number];    // x1, y1 of segment leaving this key
  easeIn:  [number, number];    // x2, y2 of segment arriving at next key
  spatialIn?: Vec2; spatialOut?: Vec2;  // position path tangents
}
```

Easing is stored per segment, between keyframe *k* and *k+1*: (x1, y1) from `k.easeOut`, (x2, y2) from `k+1.easeIn`. This matches both CSS `cubic-bezier(x1, y1, x2, y2)` and Lottie's `o` / `i` handles.

Serialization — projects save as `.motion.json` (versioned schema, `"schemaVersion": 1`), with migrations applied on load. Undo/redo uses an immutable command log; every edit is a reversible command.

Bezier easing curves

Each segment's timing is a cubic bezier from (0, 0) to (1, 1) with two editable control points; x is normalized time and y is normalized progress, exactly as in CSS.

```
B(s) = 3(1-s)^2 s\,P_1 + 3(1-s)s^2\,P_2 + s^3, \quad P_0=(0,0),\ P_3=(1,1)
```

Evaluation — for a time t in [0, 1], solve x(s) = t for s, then return y(s). The solver uses Newton–Raphson (up to 8 iterations, tolerance 1e-7), falling back to bisection when the slope is near zero; a 11-sample lookup table seeds the initial guess. This is the same approach browsers use, so preview and CSS output agree.

Constraints — x1 and x2 are clamped to [0, 1] so time stays monotonic (CSS rejects other values). y1 and y2 are free, allowing overshoot and anticipation; the graph view extends its y range to fit.

Graph editor

- Two views: *speed graph* (dy/dt per segment) and *value graph* (actual property value over time, all segments of a track).
- Drag handles; hold Shift to lock handle angle, Alt to break tangent continuity across a keyframe.
- Numeric fields for x1, y1, x2, y2, and a paste box accepting `cubic-bezier(.4,0,.2,1)` or Lottie `{o,i}` objects.
- Multi-select segments to apply one curve to many.

Presets

Preset	Value
linear	0, 0, 1, 1
ease	.25, .1, .25, 1
ease-in	.42, 0, 1, 1
ease-out	0, 0, .58, 1
ease-in-out	.42, 0, .58, 1
standard (Material)	.2, 0, 0, 1
back-out	.34, 1.56, .64, 1

Users can save custom presets to the project or to a shared library file.

Multi-dimensional properties — by default all components of a vec2 share one curve. A *separate dimensions* toggle gives `position.x` and `position.y` independent tracks and curves (exported as separate Lottie properties; flattened into sampled keyframes for CSS when curves differ).

Timeline editing and playback

The timeline shows one row per layer, expandable to one row per animated property; playback evaluates the whole scene from the model each frame, so what plays is always what exports.

Composition settings — width × height (px), frame rate (12–120 fps, default 60), duration in frames, background color, and work area (in/out) for loop playback.

Editing operations

- Add a keyframe at the playhead by changing a property (auto-key mode) or with the diamond button.
- Drag keyframes; snapping to frames, other keyframes, markers and the playhead (toggle with S).
- Box-select, then move, scale in time around the playhead or first key (Alt-drag), reverse, or copy/paste across properties of the same type.
- Change interpolation per keyframe: hold, linear, bezier (right-click or the inspector).
- Layer bars: trim in/out, slide, and offset the whole layer's keys.
- Markers with labels; exported as Lottie markers and as CSS comments.

Evaluation — for frame f on a track, find the segment $[k_i, k_{i+1}]$ containing f with binary search, compute $t = (f - k_i.\text{frame}) / (k_{i+1}.\text{frame} - k_i.\text{frame})$, apply the segment's easing, then interpolate by the property type. Before the first key the first value holds; after the last, the last value holds.

Playback — a `requestAnimationFrame` loop maps wall-clock time to frames at the composition rate; rendering uses an SVG renderer for editing fidelity and an optional Canvas 2D renderer for large scenes. Scrubbing renders on pointer move. Playback modes: once, loop, ping-pong; speed 0.25×–2×.

Onion-skinning

Onion skins draw ghosted copies of the selected layers at nearby frames, so spacing and arcs are visible without playing the animation.

Settings

Setting	Default	Range
Frames before / after	3 / 3	0–20 each

Setting	Default	Range
Step	2 frames	1–30, or "keyframes only"
Past tint / future tint	red / blue	any color
Opacity falloff	50% → 10%	linear across range
Scope	selected layers	selected, all, or pinned layers
Motion path	on	shows position path with frame ticks

Rendering — ghosts render into a separate offscreen layer beneath the live frame: each ghost is the scene evaluated at f ± n·step, drawn in outline or tinted-fill mode, composited with its falloff opacity. Ghost frames are cached by (frame, layer, content hash) and invalidated only for layers whose tracks changed, keeping scrubbing at 60 fps for up to ~200 shapes × 10 ghosts.

Motion path — for `position`, the spatial bezier path is drawn with a tick at every frame; tick spacing shows speed at a glance (dense = slow, sparse = fast). Spatial tangents are editable directly on the canvas.

Export

Both exporters map keyframes and bezier segments one-to-one where the target format can express them, sample the rest, and list every approximation in a fidelity report shown before download.

Lottie JSON

Targets the bodymovin schema (version `5.7`), playable in lottie-web, lottie-ios, lottie-android and dotLottie players.

Editor concept	Lottie field
Composition size, fps, duration	`w`, `h`, `fr`, `ip`, `op`
Shape / group / null layer	`ty: 4` / shape group `gr` / `ty: 3`
Transform properties	`ks.p`, `ks.a`, `ks.s`, `ks.r`, `ks.o`
Keyframe	`k[]` entry with `t`, `s`

Editor concept	Lottie field
Segment easing (x1,y1,x2,y2)	`o: {x:[x1], y:[y1]}`, `i: {x:[x2], y:[y2]}`
Hold keyframe	`h: 1`
Position path tangents	`to`, `ti`
Fill / stroke / path / trim	`fl`, `st`, `sh`, `tm`
Markers	`markers[]`

Colors are written as 0–1 floats; rotation in degrees; scale in percent. Text layers export as converted shape outlines by default (`text: "outlines"`) because Lottie text needs embedded font data; `text: "glyphs"` embeds glyph data instead. Output is minified by default and validated against the Lottie JSON schema before download.

CSS keyframes

Produces one `@keyframes` rule per animated element plus a matching `animation` declaration, and an HTML/SVG scaffold with class names.

```
.card { animation: card-anim 600ms linear 0s 1 both; }
@keyframes card-anim {
  0%   { transform: translate(0px, 24px) scale(0.96); opacity: 0;
         animation-timing-function: cubic-bezier(.2, 0, 0, 1); }
  60%  { transform: translate(0px, -4px) scale(1.01); opacity: 1;
         animation-timing-function: cubic-bezier(.4, 0, .6, 1); }
  100% { transform: translate(0px, 0px) scale(1); opacity: 1; }
}
```

Mapping rules

- Keyframe percentages = key frame / layer duration × 100, rounded to 3 decimals.
- Per-segment easing goes in `animation-timing-function` on the starting keyframe, which CSS applies to that segment only.
- Hold keyframes → `steps(1, end)`.
- CSS animates `transform` as one property, so transform tracks with *different keyframe times or curves* are merged: the exporter unions their key times and samples the other tracks at those times. Option `transform: "individual"` instead emits separate `translate`, `rotate`, `scale` properties (supported in all current major browsers), keeping curves exact.

- Spatial (curved) position paths, path morphs and trim paths have no CSS equivalent; they are sampled at `css.sampleRate` (default every 2 frames) with linear timing, or exported as SVG `offset-path` / SMIL where `css.allowSvg` is on.
- Optional `prefers-reduced-motion` block that jumps to the final state.

Fidelity report — lists each sampled track, key count before/after, and maximum value error versus the editor (px, degrees, or %).

API, CLI and shortcuts

The core (model, evaluator, exporters) is a framework-free TypeScript package, so the same code powers the editor, a Node CLI for build pipelines, and embedding in other tools.

Core API

```
import { Project, evaluate, toLottie, toCSS } from "@motion-tl/core";

const p = Project.load(json);                    // .motion.json
const card = p.layer("card");
card.prop("opacity").addKey(0, 0, { ease: [.2, 0, 0, 1] });
card.prop("opacity").addKey(36, 100);

const frame = evaluate(p, 18);                   // scene state at frame 18
const lottie = toLottie(p, { version: "5.7", text: "outlines" });
const { css, html, report } = toCSS(p, { transform: "individual", sampleRate:
2 });
```

CLI

```
motion-tl export lottie hero.motion.json -o hero.json
motion-tl export css    hero.motion.json -o hero.css --transform individual
motion-tl validate      hero.json                 # Lottie schema check
motion-tl report        hero.motion.json --target css   # fidelity only
```

Exit codes: 0 success, 1 fidelity error above `--max-error`, 2 invalid project, 3 I/O error.

Keyboard shortcuts

Action	Shortcut
Play / pause	Space

Action	Shortcut
Previous / next frame	, / .
Previous / next keyframe	J / K
Add keyframe on selected property	Shift + K
Toggle auto-key	A
Toggle graph editor	G
Toggle onion skin	O
Set ease preset (ease-in-out)	F9
Toggle snapping	S
Undo / redo	Cmd/Ctrl + Z / Shift + Cmd/Ctrl + Z

Testing, limitations and roadmap

Correctness is defined as the exported animation matching the editor's evaluator frame by frame within a stated tolerance.

Testing

- Unit tests for the bezier solver against reference values (including overshoot curves and near-vertical slopes).
- Round-trip: project → Lottie → import → project must be lossless for supported features.
- Frame parity: render each fixture in the editor, in lottie-web, and as CSS in headless Chromium; compare per-frame screenshots (≤ 1% pixel difference) and sampled property values (≤ 0.5 px, 0.5°, 0.5%).
- Schema validation of all Lottie output; CSS output linted and parsed.
- Performance: 60 fps playback and scrubbing for 200 animated shapes on a mid-range laptop.

Known limitations

- CSS cannot express spatial paths, path morphing or per-component easing on a shared `transform`; these are sampled and noted in the fidelity report.

- Lottie players differ in feature support (e.g. some mobile players ignore certain trim or merge-path settings).
- Color interpolation in linear sRGB can look different from browsers' default sRGB interpolation in CSS output; an option forces sRGB to match.
- Imported SVG is limited to paths, basic shapes, groups and solid fills/strokes; gradients and masks are flattened or dropped with a warning.

Roadmap

- [ ] Gradient fills and masks in both exporters
- [ ] Web Animations API (`element.animate`) export
- [ ] Spring/physics easing that bakes to bezier or sampled keys
- [ ] Lottie import from third-party files with a compatibility report
- [ ] Real-time collaborative editing